
Software Requirements Specification

for

EventHub – Campus Event Management System

Version 1.0 approved

**Prepared by Yunie Cho (s4001725), Zac Spongberg (s4096726), Agampreet
Singh (s4014941), Lucas Aponso (s3896348)**

RMIT

10/8/2025

Table of Contents

Revision History	ii
1. Introduction.....	1
1.1 Purpose	1
1.2 Document Conventions	1
1.3 Intended Audience and Reading Suggestions	1
1.4 Product Scope.....	1
1.5 References	2
2. Overall Description	2
2.1 Product Perspective	2
2.2 Product Functions.....	3
2.3 User Classes and Characteristics	5
2.4 Operating Environment	6
2.5 Design and Implementation Constraints	6
2.6 User Documentation.....	7
2.7 Assumptions and Dependencies.....	7
3. External Interface Requirements	8
3.1 User Interfaces.....	8
3.2 Hardware Interfaces	11
3.3 Software Interfaces.....	12
3.4 Communications Interfaces.....	12
4. Nonfunctional Requirements	13
4.1 Performance Requirements	13
4.2 Safety Requirements	13
4.3 Security Requirements	14
Software Quality Attributes	14
4.4 Business Rules.....	15
5. Other Requirements	15
6. System Architecture.....	16
6.1 Architecture Overview	16
6.2 Architectural Decisions.....	17
6.3 Approaches to Achieve Quality Goals	18
7. User Interface Design	19
8. Appendix A: Glossary.....	26
9. Appendix B: Analysis Models	26
Page Structure	26
10. Appendix C: Milestone 2	28
10.1 New Requirements in Milestone 2	28
10.2 Description of New Functionality in Milestone 2	28
10.3 Code Files Added in Sprint 2	29
10.4 Test Files Added in Sprint 2.....	30
10.5 Other Files	31

Revision History

Name	Date	Reason For Changes	Version

1. Introduction

1.1 Purpose

EventHub's purpose is to provide a user-friendly web platform that streamlines the management and participation of university events for both students and organisers. It allows university clubs and student organisers to create, publish, and manage events. Students can browse upcoming events, RSVP, and get reminders, and club organisers can track attendance of events and gather feedback from users.

1.2 Document Conventions

- **Bold text** indicates section headers and important terminology
- *Italic text* indicates placeholder information or emphasis
- Requirements are uniquely identified with format FR-XXX (Functional Requirements) and NFR-XXX (Non-functional Requirements)
- Priority levels: High (essential), Medium (important), Low (desirable)
- Priorities for higher-level requirements are assumed to be inherited by detailed requirements.

1.3 Intended Audience and Reading Suggestions

This document is intended for:

- **Development team:** Focus on Sections 2-4 for implementation details
- **Project Managers:** Review Sections 1,2, and 5 for scope and constraints
- **Quality Assurance:** Emphasize Sections 2-4 for testing requirements
- **Client/Stakeholders:** Read Sections 1,2, and 7 for system overview and UI design
- **System Administrators:** Focus on Sections 4 and 6 for deployment and architecture

Start with Sections 1 for context, then proceed to Section 2 for system overview, then focus on sections most relevant to your role.

1.4 Product Scope

EventHub is a comprehensive web-based event management system designed for university environments. The system serves three primary user classes: students, event organisers, and administrators. The system is composed of two primary layers: a client-facing web application that delivers a responsive and interactive user experience, and a server-side application that manages business logic, data processing, and integration with campus systems. EventHub's features can be grouped into core features, which provide essential event management and participation functionality, and extended features, which enhance engagement and offer additional value to students and administrators.

Core Features:

- Event Management: Creating, editing, viewing, and deleting events.
- User Participation: RSVP functionality and attendance tracking.
- Notifications: Alerts and reminders for upcoming or relevant events.
- User Authentication: Secure login and session management.
- Administrative Controls: User management (editing details/deleting users) and event moderation (editing and removing events).

Extended Features:

- User Participation (Extended): Event sharing and feedback collection.
- Event Recommendations: Personalized event suggestions based on user behavior and history.
- Gallery Features: Post-event media uploads for community viewing.

Key Business Objectives

- Centralize campus event information and management
- Increase student participation in university activities through accessible discovery and engagement tools.
- Provide insight for event organisers

The platform will integrate with campus data and support user authentication and session management. It is based on a relational database that contains event and club management, comparison, and recommendation functions, as well as the ability to notify users about relevant events, clubs, or payments.

1.5 References

- RMIT SRS Document Template
- *MySQL :: MySQL 8.4 Reference Manual :: MySQL Glossary. (2024). Mysql.com.*
<https://dev.mysql.com/doc/refman/8.4/en/glossary.html>
- *Oaic. (2023, October 9). Australian privacy principles. OAIC.*
<https://www.oaic.gov.au/privacy/australian-privacy-principles>

2. Overall Description**2.1 Product Perspective**

EventHub is a new, self-contained web application designed specifically for the RMIT university environment. It may integrate with existing systems, such as:

- Student Authentication Services
- Campus Data Sources for club and event location information
- Third-Party APIs (e.g., Gmail API for email services)

For a high level view of the major components of the overall system, subsystem interconnections, and external interfaces, refer to the Architectural Diagram (Section 6.1) provided below

2.2 Product Functions

Event Creation

- FR-001: The system shall allow organisers to create new events with the following compulsory fields:
 - Event title (text field, max 200 characters)
 - Event description (rich text editor, max 2000 characters)
 - Date and Time (date/time picker with validation for future dates)
 - Location (text field)
 - Keyword/tags (multi-select from predefined tags)
- FR-002: The system shall validate event data to ensure all required fields are completed before event submission
- FR-003: The system shall automatically assign the logged-in user as the event organiser upon creation
- FR-004: The system shall generate a unique event ID upon successful creation.

Event Modification

- FR-005: The system shall allow event organisers to modify event details except event ID
- FR-006: System shall notify all RSVP'd users of significant changes (date, time, location)
- FR-007: System shall provide organisers with the ability to delete events
- FR-008: System shall provide a confirmation dialog prior to the deletion of events
- FR-009: System shall restrict edit/delete permissions to original organiser only

Event Display and Discovery

- FR-010: Home page shall display upcoming events in chronological order (next 5) with an option to view more
- FR-011: System shall provide a detailed event listing, including:
 - Event title
 - Date and Time
 - Location
 - Event description
- FR-012: The system shall provide filters that allow users to refine searches by tags
- FR-013: The system shall provide sort functionality to display results in:
 - Sort by date (default: closest)
 - Sort by Name

User Participation

- FR-014: The system shall allow logged-in users to RSVP to any upcoming event by clicking an RSVP button on the event detail page.
- FR-015: The system shall prevent duplicate RSVPs from the same user for the same event.

- FR-016: The system shall allow users to cancel their RSVP at any time before the event's scheduled start.
- FR-017: The system shall provide users with a personalized dashboard that displays a list of all upcoming events they have RSVP'd to.
- FR-018: The system shall allow users to receive automated reminders (via email or in-app notification) at a user-specified time before an event they have RSVP'd to.
- FR-019: The system shall allow organisers to view a list of users who have RSVP'd to their events from the organiser dashboard.
- FR-020: The system shall restrict RSVP functionality to authenticated users only.
- FR-021: The system shall provide an event feedback form (rating + comments) for event participants after the event.
- FR-022: The system shall provide a "share" feature to allow users to share event link.

Event Recommendations

- FR-023: The system shall generate event recommendations for each logged-in user on the home page or dashboard.
- FR-024: The system shall recommend events based on the categories and tags of events the user has previously RSVP'd to.
- FR-025: The system shall recommend events organized by clubs the user has joined or interacted with.
- FR-026: The system shall identify similar users based on overlapping RSVP history and recommend events those users have RSVP'd to.
- FR-027: The system shall display recommended events in a separate "Recommended for You" section, limited to a maximum of 5 events at a time.
- FR-028: The system shall update event recommendations dynamically whenever the user RSVPs to a new event or joins a new club.
- FR-029: The system shall exclude events the user has already RSVP'd to or events that have already occurred from the recommendation list.

Event Gallery

- FR-030: The system shall allow organisers to upload photos after an event to create a gallery visible on the event detail page.
- FR-031: The system shall restrict gallery upload permissions to event organisers only.

Admin Tools

- FR-032: The system shall provide administrators with the ability to view and manage all events
- FR-033: The system shall provide administrators with the ability to delete inappropriate events.
- FR-034: The system shall provide administrators with user management capabilities, including deactivation and banning

Extension/Custom Features

- FR-035: The system shall send email confirmations for all RSVP actions (confirmation. Cancellation
- FR-036: The system shall provide notification panel showing new events matching user interests
- FR-037: The system shall send automated email reminders 24 hours before events

- FR-038: The system shall send RSVP confirmation emails to users after RSVPing
- FR-039: The system shall provide a notification panel

Club Management

- FR-040: The system shall provide a club profile page displaying club name, description, tags, and list of upcoming events.
- FR-041: The system shall display a list of all clubs, filterable by tags.

Notifications and Reminders

- FR-042: The system shall provide both in-app and email notifications for new events, reminders, and updates.
- FR-043: The system shall allow users to customize notification preferences (email only, in-app only, both).

2.3 User Classes and Characteristics

EventHub will be used by 3 types of users, each with different levels of access and technical expertise. The system's design must account for the varying needs and privileges of these groups.

1. Students (General Users)

This is the largest user group, consisting of university students who attend events. Their primary product functions include:

- Browsing upcoming events
- Filtering and searching events by date, category, or keywords
- RSVP and cancellation of RSVP
- Receiving reminders and notifications
- Providing feedback after events

Students will have a moderate frequency of usage and interaction with the software. Their technical expertise includes basic computer and web navigation skills, and their security level is the lowest; they can only interact with public event information and their own account data. This user class is the most important to satisfy, as they are the biggest class whilst also possessing the least knowledge of the system, making frustration with the software more likely.

2. Club Organisers (Event Creators)

This the second largest group, consisting of members of university clubs or societies who will create and manage events. Their primary product functions include:

- Event creation, editing, and deletion
- Viewing RSVP lists for their own events
- Uploading post-event photos to the gallery

- Viewing and responding to event feedback

Club Organisers have a moderate to high frequency of usage and interaction with this software. Their expected technical expertise requires them to understand and use basic media upload tools, as well as basic computer and web navigation skills. They possess elevated privileges for managing only their own events and related data. This classes' importance is high and should not be overstated, as their participation facilitates the participation and benefits of the system for the Student user class.

3. System Administrators (Moderators)

Description: Designated university staff or authorised personnel responsible for system oversight and moderation.

This is the smallest user group, consisting entirely of designated university staff or authorised personnel responsible for system oversight and moderation. Their primary product functions include:

- Managing all events (edit/delete)
- Managing user accounts (activate, deactivate, or ban)
- Removing obscene content or images
- Overseeing platform compliance with university policies

System Administrators have a high frequency of usage and interaction with this software to facilitate their ongoing moderation and management of the system. Their expected technical expertise is high; they must be comfortable with advanced administrative interfaces and moderation tools. They possess the highest privilege level, allowing them full access to system data and administrative controls. As this class has the most understanding and familiarity with the system alongside the highest technical expertise, this class is the least important to satisfy. UI and visual design for this class should not be prioritized; however, it is important that this class not be overlooked, as their experience with the system affects the rest of the user base.

2.4 Operating Environment

Server Environment

- Containerization & Deployment: Docker with support for CI/CD pipelines.
- APIs: RESTful services using JSON over HTTPS.

Client Environment

- Hardware: Any modern desktop, laptop, tablet, or mobile device.
- Operating System: Cross-platform support for Windows 10+, macOS, iOS, and Android.
- Frontend Application: Java and Thymeleaf running in the browser.
- Web Browsers: Google Chrome, Mozilla Firefox, Safari, and Microsoft Edge.

Network Requirements: Stable internet connection (minimum 1 Mbps downstream) to access the system.

2.5 Design and Implementation Constraints

Technology Stack

- The application must be developed using Spring Boot (Java) for backend services.
- The system must use MySQL as the relational database management system.
- The system must be deployable via Docker containers, ensuring consistent runtime environments across development, staging, and production.
- The system must use Java and Thymeleaf for its frontend/visual services.

Development Process

- The project must follow Agile methodology, with iterative development cycles, sprint planning, and continuous stakeholder feedback.
- Codebase must follow object-oriented design principles and clean code standards for maintainability.

Security & Compliance

- The system must comply with Australian Privacy Principles (APPs) regarding student data handling and storage.
- All data transmitted between client and server must use HTTPS/TLS encryption.
- Authentication must be implemented using secure token-based methods (e.g., JWT).

Integration & Interfaces

- The system must expose RESTful APIs for integration with external systems.
- The frontend must interface seamlessly with backend APIs via JSON over HTTPS.

Operational Constraints

- Hardware requirements for deployment must support at least 4 CPU cores, 8 GB RAM, and 100 GB storage.

2.6 User Documentation

The following user documentation components will be delivered along with the Student Event Management System:

User Manual

- A written guide that provides step-by-step instructions on how to use the system's core features.
- The manual will include sections for different user roles (Student, Administrator, and Organiser), covering account setup, event management, RSVP, and administrative controls.
- The manual will be delivered in PDF format and made accessible within the application

2.7 Assumptions and Dependencies

Assumptions

- Users will have valid RMIT credentials (e.g., student or staff login) to authenticate into the system.
- Users will have access to a modern web browser (e.g., Chrome, Firefox, Safari, or Edge) with JavaScript enabled.

- Event organisers and administrators are assumed to have basic technical knowledge for creating and managing events.
- The Google API is in working order.
- This system shall only be used within Australia.

This system has no external dependencies from other projects.

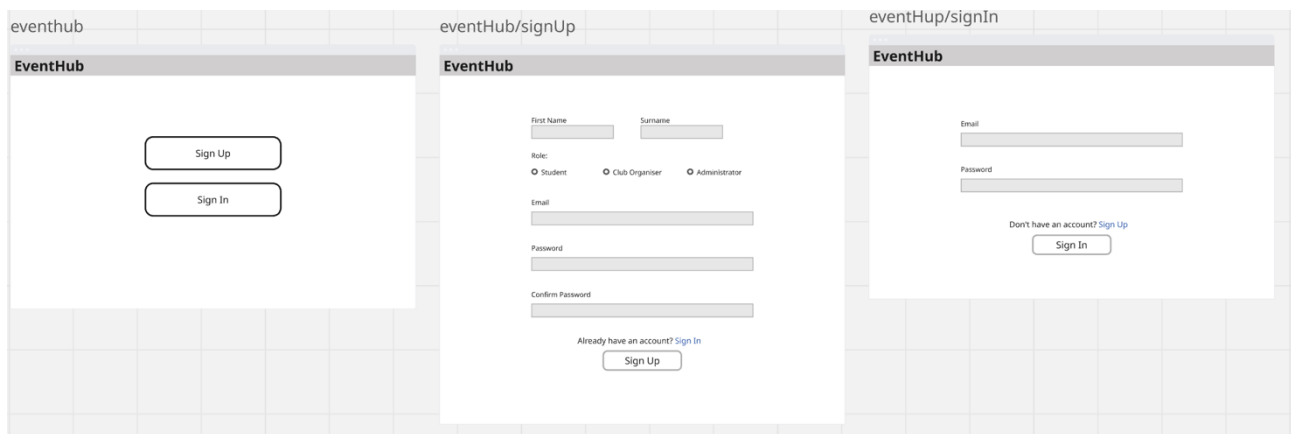
3. External Interface Requirements

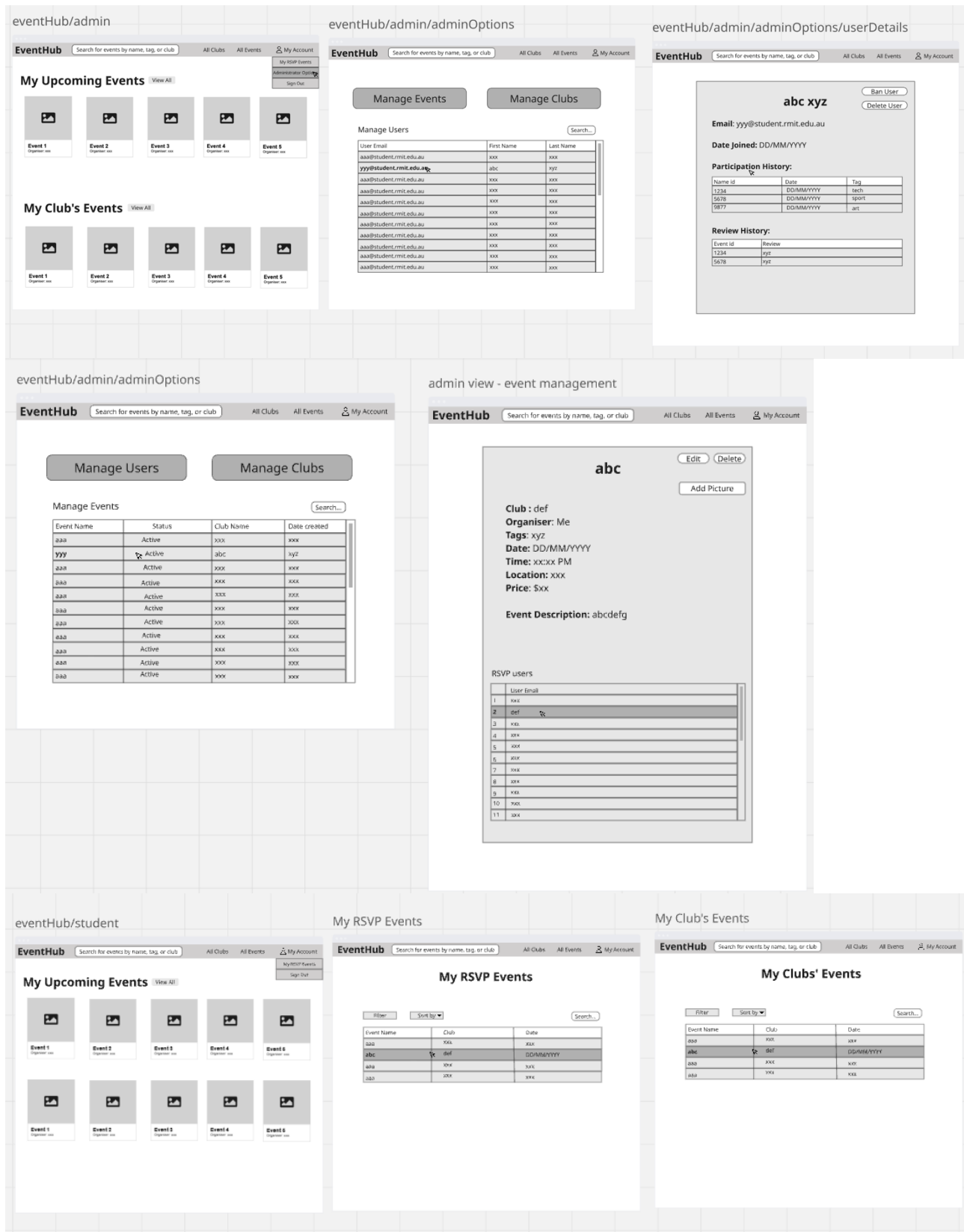
3.1 User Interfaces

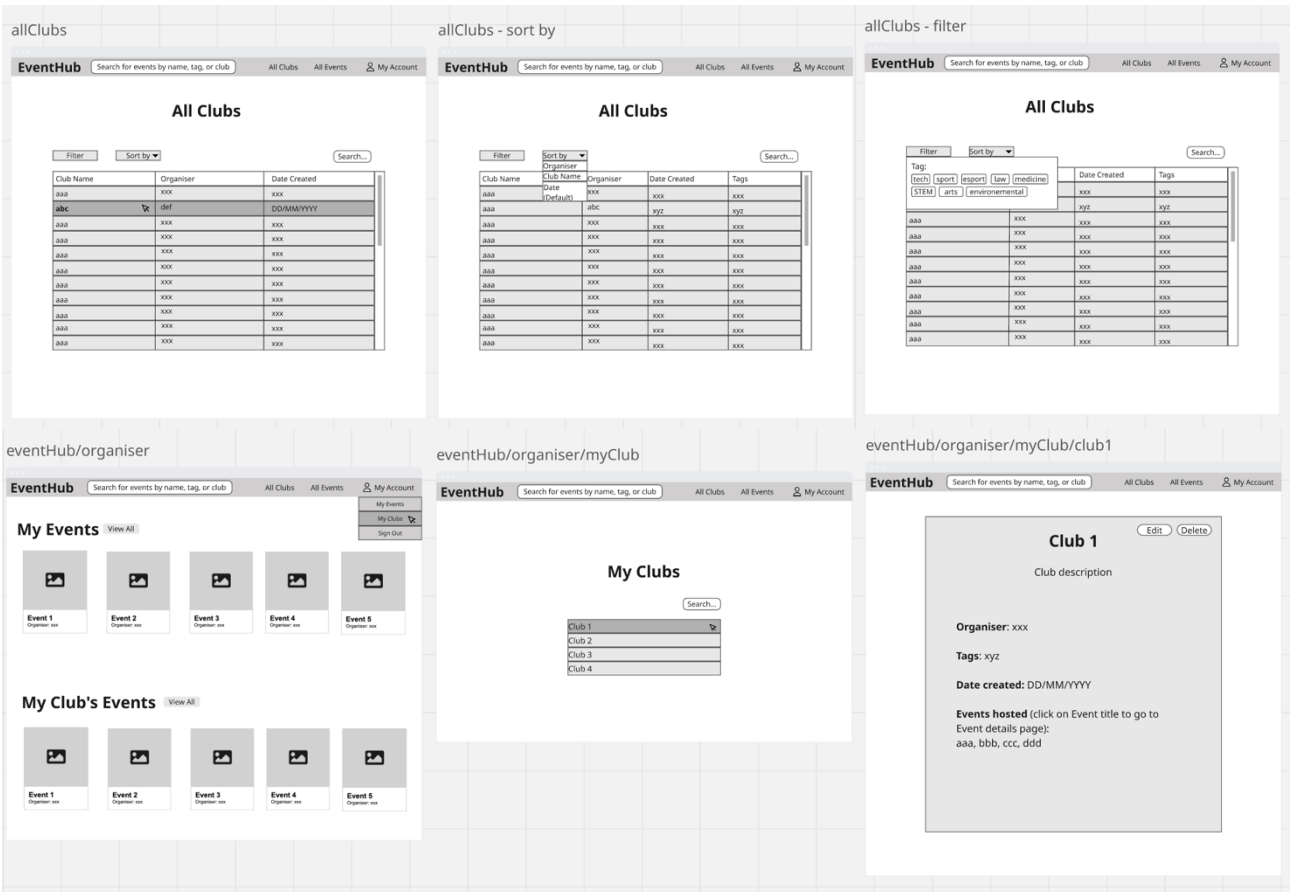
The system's user interface shall be web-based and accessible via modern web browsers (Chrome, Firefox, Safari, and Edge). The UI will be modern and responsive, adapting to desktop, tablet, and mobile devices.

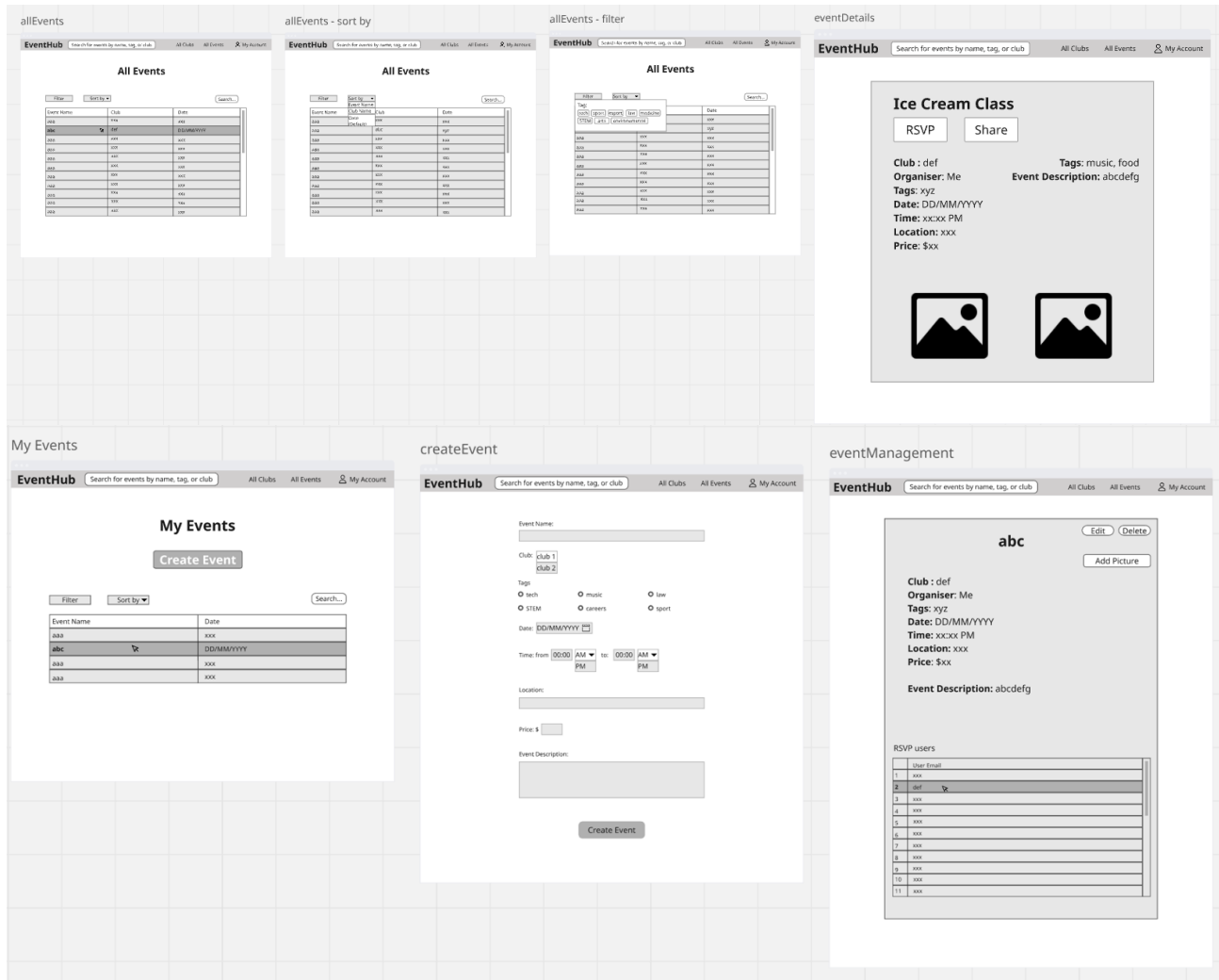
Logical Characteristics:

- Navigation: A unified navigation bar with links to Home, Events, Clubs, Profile and Admin Dashboard (admins only).
- Standard Functions: Each screen will include standard buttons for Save, Cancel, and Back.
- Error Handling: Errors (such as a failed login or an incorrect RSVP) will be shown in non-intrusive modals or inline messages with detailed descriptions.
- Forms will employ conventional UI features like ,dropdowns, multi-select fields (for tags), date/time pickers (for events), and file uploads (for event galleries).
- Sample Screens: Event creation form (title, description, date/time, location, category, tags), user dashboard (list of RSVP'd events), and club site (club information and events).









3.2 Hardware Interfaces

The system is designed to be a web application, therefore necessitating only normal computing devices without the need for specialist hardware.

- **Compatible Devices:**
 - Desktop and laptop computers (Windows, macOS, Linux).
 - Tablets and smartphones (Android, iOS operating systems).
- **Interfaces:**
 - User devices will communicate through web browsers using HTTPS.
 - The backend and database are installed using Docker containers on Linux servers.
- **Data and Control Interactions:**
 - User activities, such as form submissions and RSVPs, are executed through RESTful API calls.
 - The database is accessible exclusively by the backend using the JDBC connector.

3.3 Software Interfaces

The system integrates with the following software components:

- **Backend:** Java 17 with SpringBoot framework for REST API.
- **Frontend:** Java + Thymeleaf templates for rendering UI.
- **Database:** MySQL (latest stable version).
- **Build Tools:** Maven for backend dependency management.
- **Containerization:** Docker for all system components.
- **CI/CD:** GitHub Actions for automated testing and deployment.
- **External Libraries/Tools:**
 - Google API (for integration with Gmail notifications)
 - Spring-Mail (for delivering RSVP confirmations and reminders).
 - Thymeleaf (server-side templating engine).
- **Data Sharing:**
 - REST API endpoints will return JSON replies.
 - Shared data includes user details, event info, RSVPs, feedback, and club info.
 - Database schema defines relationships for shared access across backend logic.

3.4 Communications Interfaces

The system requires dependable communication interfaces to facilitate user interactions, notifications, and integrations:

Protocols:

- HTTP/HTTPS: Used for all web-based interactions between frontend and backend.
- JDBC: Used for backend-to-database connectivity.

Email Communications:

- RSVP confirmations, reminders, and notifications will be delivered over SMTP.
- Email message formatting will use HTML templates (for better readability).

Security:

- All external communication will use HTTPS with TLS encryption.
- Authentication will follow safe login methods (hashed passwords, session management).

Performance Requirements:

- Data transfer speeds optimised for web interactions (<200ms response for common requests).
- Event listings and dashboards should load within 2 seconds under normal load.
- Synchronization:
- Frontend and backend communications follow the request-response model.

- To load data more effectively, the system will use Async/Await methods to load data asynchronously.

4. Nonfunctional Requirements

4.1 Performance Requirements

- **NFR-01:** The system shall support at least 500 concurrent student users without significant performance degradation. **Rationale:** This ensures scalability to handle peak times such as event RSVP openings or ticket price changes.
- **NFR-02:** 95% of page requests shall load within 2 seconds under normal load and within 5 seconds under peak load. **Rationale:** One of the main goals of this software is to streamline user experience with event participation and management, and the speed of the software is an essential aspect of this.
- **NFR-03:** The system shall maintain daily uptime of at least 99%, excluding planned maintenance. **Rationale:** This makes sure users can reliably access the system for time-sensitive activities.
- **NFR-04:** The system shall be deployable using Docker containers to simplify setup and demonstrate portability across environments. **Rationale:** This allows for easy testing, deployment, and scaling across development, staging, and production.
- **NFR-05:** The system shall process RSVP submissions within 1 second under normal load and within 3 seconds under peak load. **Rationale:** This is incredibly important because RSVP'ing is a time-sensitive feature where students expect immediate confirmation to secure their event spots.
- **NFR-06:** Notifications for upcoming events shall be generated and delivered to users within 5 seconds of the scheduled trigger time. **Rationale:** This makes sure notifications are timely and effective.

4.2 Safety Requirements

- **NFR-07:** System failures (e.g., database outage) shall trigger graceful error messages without exposing sensitive technical details such as stack traces, database errors, or configuration details. **Rationale:** This prevents confusion and protects users from accidentally misusing the system whilst safeguarding sensitive information.
- **NFR-08:** The system shall recover gracefully from server restarts to make sure data is persistently stored in the database by preventing any memory-only storage for important data such as RSVP submissions. **Rationale:** Prevents data loss or corruption when the system encounters unexpected shutdowns.
- **NFR-09:** The system shall support database backup and restore functionality. **Rationale:** Allows the system to recover from data corruption or catastrophic failures.

- **NFR-10:** The system shall automatically log out users after 15 minutes of inactivity to prevent unauthorized access on shared devices. **Rationale:** Protects users from misuse if they forget to log out in public spaces (e.g., university labs).
- **NFR-11:** The system shall provide mechanisms to safeguard users from harmful or inappropriate content by including reporting features, with reports stored and retrievable for administrative review. **Rationale:** Ensures user well-being by addressing harmful or offensive material.

4.3 Security Requirements

- **NFR-12:** The system shall use JWT-based authentication to manage session validity securely. **Rationale:** This makes sure that compromised tokens cannot be reused indefinitely, which promotes the security of the system.
- **NFR-13:** The system shall only collect necessary personal details and restrict their visibility. **Rationale:** This reinforces users privacy and trust in the system.
- **NFR-14:** The system shall use role-based access control (RBAC) to ensure that:
 - Students can RSVP and manage their own profiles.
 - Organisers can manage only their own clubs/events.
 - Administrators can manage all users, clubs, and events. Only administrators can view personal user details.
 - **Rationale:** This enforces the principle of least privilege, reducing security risks by ensuring that users can only access the resources necessary for their role.
- **NFR-15:** All communication shall use HTTPS/TLS 1.2 or higher to protect data in transit. **Rationale:** This ensures that sensitive information is encrypted and prevents interception or tampering by unauthorised parties.
- **NFR-16:** The system shall log all security-relevant events (failed login attempts, password resets, privilege changes, etc.) and make them available for admin review. **Rationale:** This makes detection of malicious activity and misuse easier to detect and address.

Software Quality Attributes

- **NFR-17:** The system shall provide a consistent UI across all pages with navigation accessible within 3 clicks from the homepage. **Rationale:** This improves usability, reduces cognitive load and allows users to quickly find and access key features.
- **NFR-18:** The system codebase shall follow a modular design with clear internal documentation. **Rationale:** This simplifies debugging, enhances maintainability, and ensures that future updates can be made with minimal disruption.
- **NFR-19:** The system shall support automated tests covering core functionality to reduce regressions during development. **Rationale:** Automated testing improves reliability for previously defined features.
- **NFR-20:** The system shall validate all form inputs (e.g., event creation, RSVP, registration) at both the client and server side with 100% accuracy. **Rationale:** This prevents incorrect or inconsistent data from entering the database and increases the integrity of the system.

- **NFR-21:** The system shall be accessible and fully functional across the latest versions of Chrome, Firefox, Safari, and Edge browsers. **Rationale:** This ensures availability and usability by supporting the browsers users are likely to use and avoiding compatibility issues.
- **NFR-22:** The system shall be designed to scale horizontally to support a 100% increase in concurrent users without more than a 20% performance degradation. **Rationale:** This promotes scalability and adaptability, allowing the system to grow as usage demand increases.

4.4 Business Rules

- BR-01: Only students can RSVP to events.
- BR-02: Club organisers can create and manage events only within their assigned clubs.
- BR-03: Administrators can create, edit, and delete users, clubs, and events without restrictions.
- BR-04: Each event must be associated with exactly one club.
- BR-05: Users may not RSVP to past events.

5. Other Requirements

Database Requirements:

- The system must use MySQL database with compliance for ACID.
- Foreign key constraints must ensure referential integrity (e.g., event must belong to a valid club).

Internationalization Requirements:

- Date/time formatting shall adapt to the user's local time.

As per our assumption that this software will be used within Australia, this section is limited.

Legal & Privacy Requirements:

- The system must comply Australian Privacy Principles (APPs).
- The system must allow personal data to be deleted upon account termination.

Reuse Objectives:

- The system architecture (SpringBoot backend + Thymeleaf frontend + Docker containers) shall be reusable for other university event/club systems.
- Core components like RSVP, Notifications, and Event Management should be modular for reuse in future projects.

Reliability & Recovery:

- Recovery Time Objective (RTO) must be <2 hours for critical failures.

Deployment Requirements:

- All systems shall run in Linux server environments with Docker containers.
- The system must use CI/CD pipelines (GitHub Actions) for automated testing and deployment.

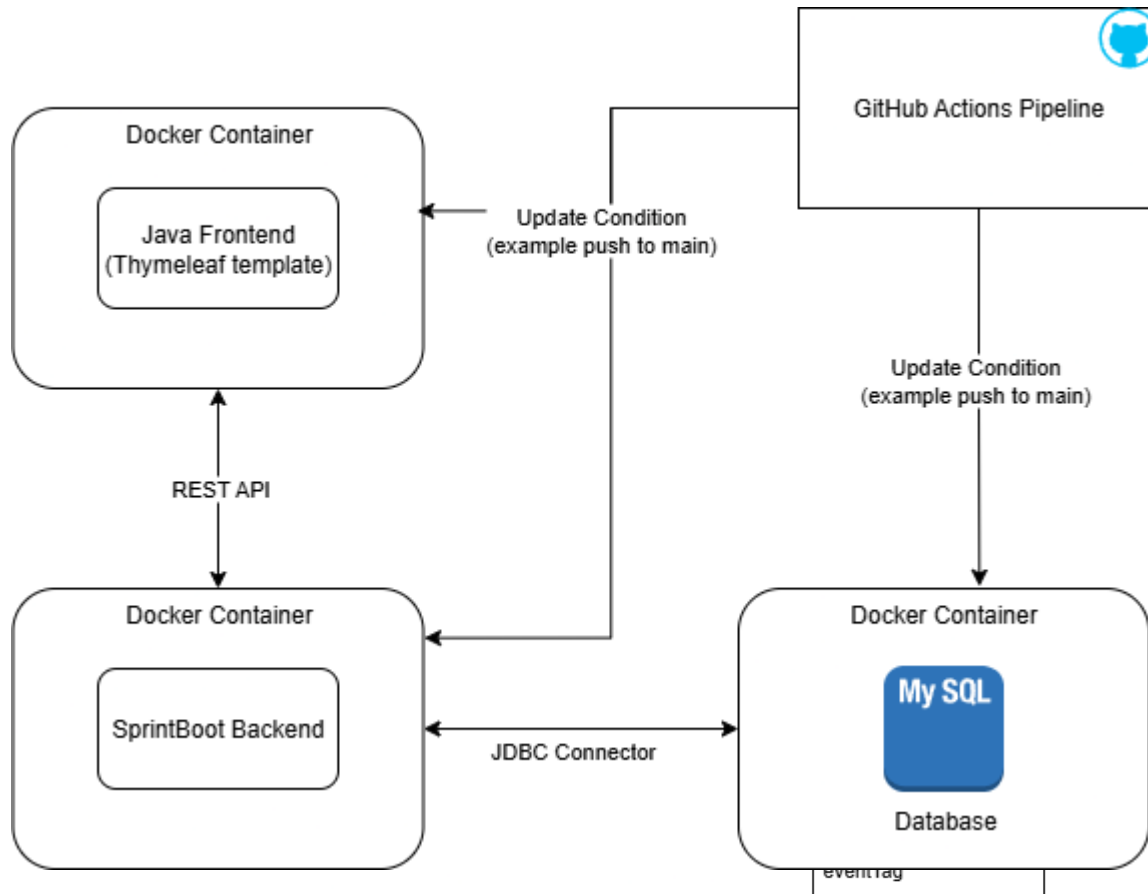
6. System Architecture

6.1 Architecture Overview

The system follows an MVC structure.

It uses various technologies like -

- Backend: Java 17, SpringBoot (Rest API)
- Frontend: Java, Thymeleaf
- Database: MySQL
- Build Tool: Maven
- Containerization: Docker
- CI/CD: GitHub Actions



6.2 Architectural Decisions

- **Backend:** Java 17 with SpringBoot (REST API)
 - **Decision:** We chose SpringBoot for its mature ecosystem, ease of building REST APIs, and integration with databases.
 - **Rationale:** Minimizes boilerplate, strong community support, rapid prototyping.
- **Frontend:** Java with Thymeleaf
 - **Decision:** Java with Thymeleaf templates for the frontend UI. This ensures consistency across the technology stack, since we are already using Java and SpringBoot in the backend.

- **Rationale:** Easier integration with SpringBoot, reducing the need for a separate JavaScript framework as well as, it simplifies testing and debugging because both backend and frontend rendering logic share the same language and ecosystem.
- **Database: MySQL**
 - **Decision:** Chosen as relational DB with high reliability and ACID properties.
 - **Rationale:** Well-suited for structured event/club/user data. Much supported by JDBC.
- **Build Tools: Maven**
 - **Decision:** Maven for backend and frontend dependency management.
 - **Rationale:** Chosen for the best alignment with its environment which would be Maven for springboot.
- **Containerization: Docker**
 - **Decision:** All components run in Docker containers.
 - **Rationale:** Ensures portability, consistency across dev/test/prod environment, and easier CI/CD automation.
- **CI/CD: GitHub Actions**
 - **Decision:** Use GitHub Actions to automate builds, tests, and deployments.
 - **Rationale:** Integration with repository, supports multi-container workflows, free of use.

6.3 Approaches to Achieve Quality Goals

- **Scalability:** Decoupled MVC layers and containerized services allow independent scaling of backend, frontend, and database.
- **Maintainability:** Modular code structure with Maven dependency management and MVC separation improves testability and long-term maintenance.
- **Reliability:** CI/CD pipeline with GitHub Actions ensures automated testing and safe deployments.
- **Security:** Role-based access control, secure authentication, and controlled database access.
- **Portability:** Docker containers guarantee consistent environments across local machines, staging, and production servers.

6.3.1 Organizational Decisions

- Team follows Agile (Scrum) for iterative progress.
- GitHub Projects used for backlog and sprint management.
- GitHub Actions ensures automated integration aligned with organizational DevOps practices.

7. User Interface Design

The EventHub user interface is designed to support a clean, intuitive, and role-based experience for the different user classes. The interface consists of a set of interconnected pages and menus that allow users to browse, create, and manage events, interact with clubs, and access personalized information.

Navigation is structured around a central banner, which is accessible from most pages and provides quick access to global features such as search, event listings, club listings, and account settings. Depending on the user's role, the banner adapts to provide additional options such as "My Clubs" for organisers or "Administrator Options" for system administrators.

The primary landing page serves as the entry point for new users, offering signup and login functionality. Once authenticated, users are directed to a homepage that highlights key information through dynamic carousel views. For students, this includes RSVP'd and recommended events, while organisers and administrators see additional content specific to their roles.

Supporting pages, such as the All Events and All Clubs pages, provide searchable and filterable tables that allow users to quickly locate events or clubs of interest. Details pages (e.g., Event Details, Club Details, User Details) present comprehensive information and offer role-based management options where applicable.

Throughout the interface, graphical components such as buttons, tables, carousels, and dropdown menus are arranged consistently to ensure ease of use and quick understanding. The design emphasizes accessibility, responsiveness, and minimal cognitive load, ensuring that both occasional users (students) and frequent users (organisers and administrators) can efficiently complete their tasks.

Landing Page

When a user first opens EventHub, they are presented with the landing page. This page provides a simple entry point into the system, offering two primary options: Sign Up (for new users) and Sign In (for returning users). From here, users can either create a new account or sign in to access the homepage and begin engaging with events, clubs, or administrative functions depending on their role.

Signup Page

The signup page allows new users to create an EventHub account. The page contains input fields for essential information such as name, email address, password, and role (student, club organiser, or administrator if applicable). Upon submission, users are redirected to the homepage with access to features tailored to their role.

Login Page

The login page enables existing users to authenticate and gain access to EventHub. Users provide their registered email and password. Successful login directs them to the homepage, where their role-specific dashboard and event recommendations are displayed.

Homepage

Once logged in, users are taken to the homepage. At the top, a banner provides universal navigation options, including the search bar, links to All Events, All Clubs, and an Account dropdown.

- For students, the page features carousels of the top 5 earliest RSVP'd events and top 5 recommended events.

- For organisers, the homepage additionally displays the top 5 upcoming events from their clubs, alongside the student-specific carousels.
- For administrators, the homepage grants access to administrative functions via the account dropdown.

The homepage serves as the central hub for discovering, managing, and navigating events and clubs within the platform.

Banner (Global Navigation)

The banner is visible on all pages except the landing, login, and signup screens. It provides consistent navigation across the system. The banner includes a search bar, a clickable logo that redirects to the homepage (escape hatch), and links to All Events and All Clubs. On the right-hand side, the Account dropdown allows users to access their RSVP'd events, profile-related options, and logout. Club organisers and administrators see additional role-specific options within this menu.

Account Dropdown Menu

Accessible from the banner, the account dropdown provides quick access to personalized features:

- Students see a link to their RSVP'd events and a logout option.
- Club organisers additionally see "View My Clubs," giving direct access to their assigned clubs.
- Administrators see "Administrator Options," providing entry to system-level management functions.

My Clubs Page

This page lists all clubs that the logged-in organiser is responsible for. Each club is presented in a table or list format, and selecting a club opens its Club Details page. This page enables organisers to efficiently manage and navigate between multiple clubs under their control.

Administrator Options Page

The administrator options page centralizes system management tasks. It contains a table of all users, where selecting a user opens the User Details page. Additional admin functions, such as managing clubs and events, can be accessed from this page. This section is restricted exclusively to system administrators.

User Details Page

Accessible only to system administrators, the user details page displays individual user information, such as name, email address, and date joined. It also includes all events the user has RSVP'd to and all the reviews they have posted. The page also includes administrative actions, such as a Delete User button, which returns the administrator to the Administrator Options page upon completion.

All Events Page

This page provides a searchable and filterable list of all events in the system. Events are displayed in a table, with columns for attributes such as name, date, club, and location. Users can filter, sort, and search to quickly locate specific events. Selecting an event opens the Event Details page.

Event Details Page

The event details page shows comprehensive information about a selected event, including its name, description, date, club, and location.

- Students can RSVP to events from this page.
- Club organisers and administrators have additional options to edit or delete event details.

All Clubs Page

This page mirrors the functionality of the All Events page but for clubs. Users can view, search, and filter through a table of all registered clubs. Selecting a club opens the Club Details page.

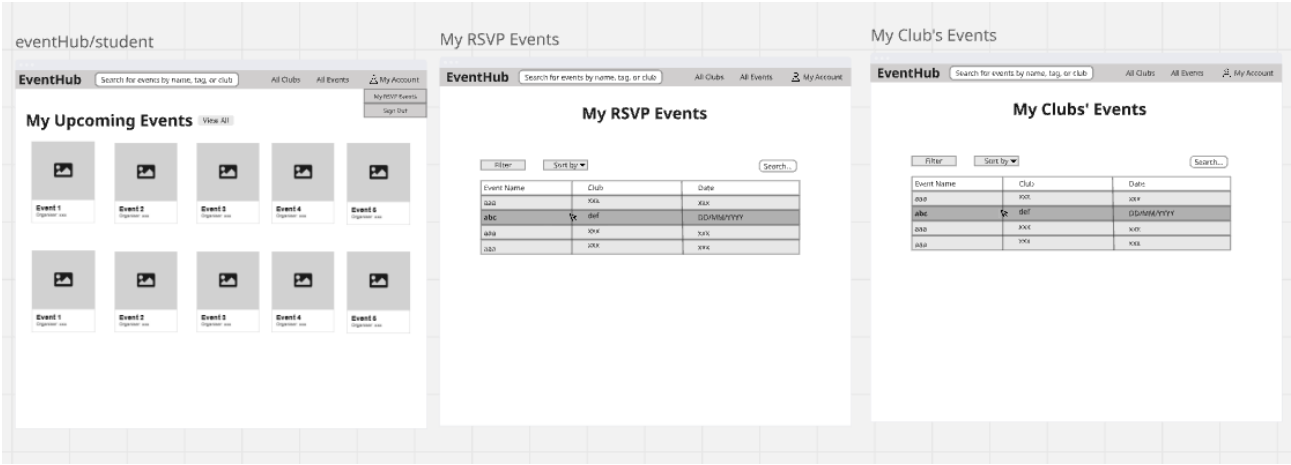
Club Details Page

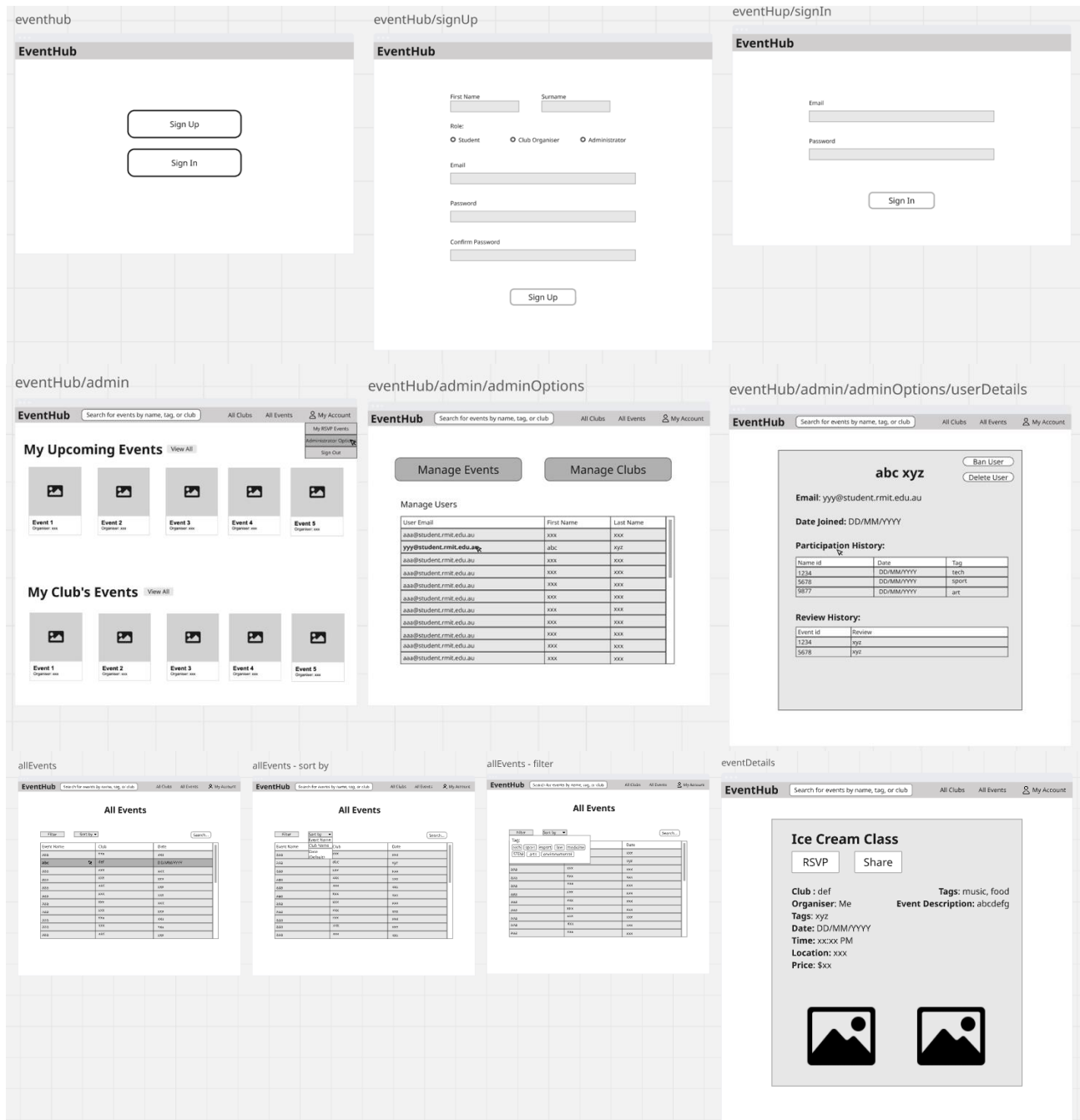
The club details page displays information about a specific club, including its name, description, organiser, tags, and date created. It also lists all events hosted by the club, with links to their respective event details.

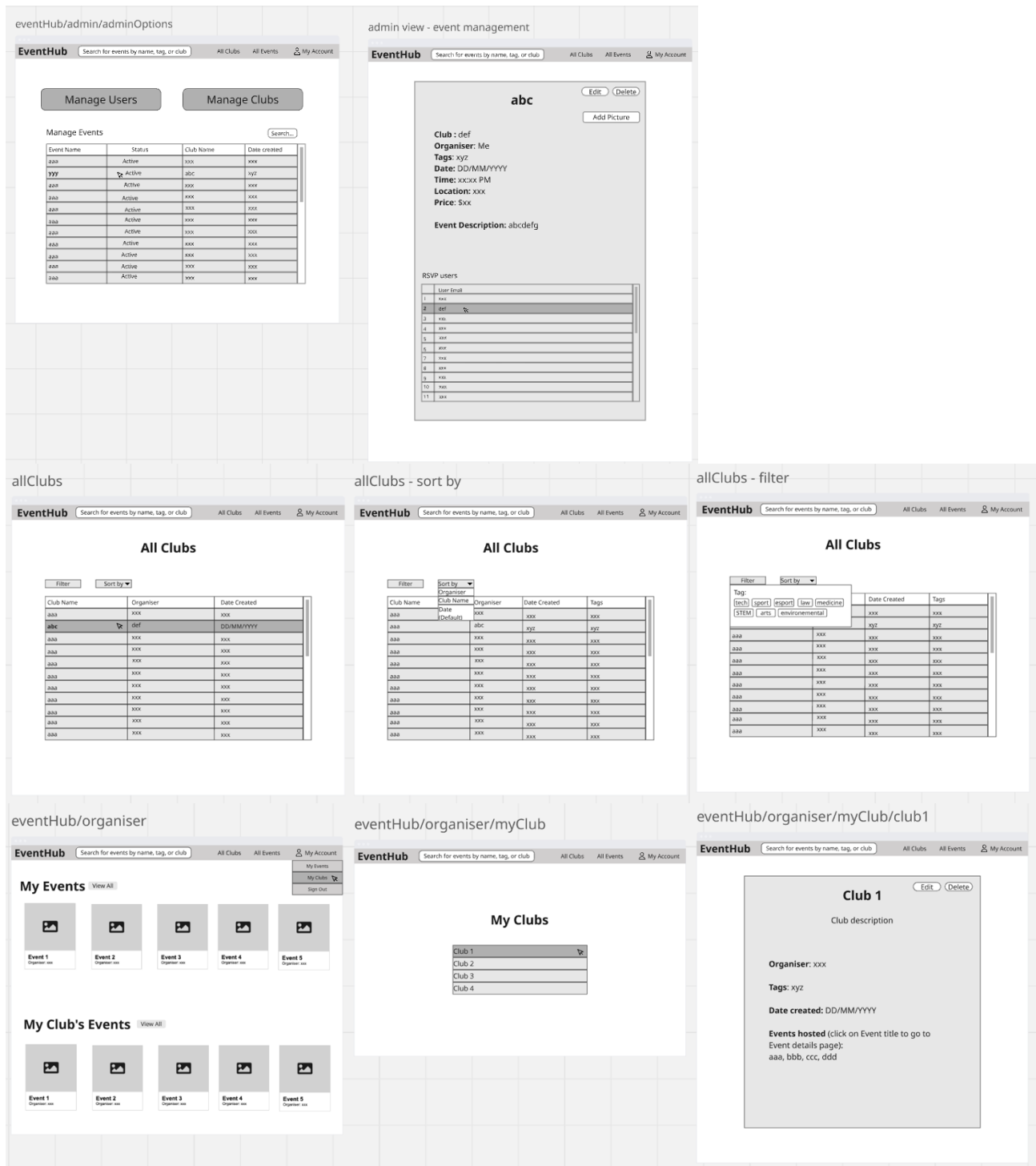
- Organisers can manage club information, including editing details or deleting the club. They can also access the 'Event Details' page via clicking on an event in the event list, and a button to access the 'Create Event' page, which is located above the event list.
- Administrators may also exercise club management privileges when necessary.

Create Event Page

The Create Event page displays interactive elements that allow authenticated users to select and adjust all elements of a new event, including text boxes, buttons, and dropdown menus. Students should not have access to this page. Selecting the 'Create Event' button at the bottom of the page will create the new event with the selected attributes and transfer the user back to the Club Details page.







My Events

EventHub

Search for events by name, tag, or club

All Clubs

All Events

My Account

My Events

Create Event

Filter

Sort by ▼

Event Name	Date
aaa	xxx
abc	DD/MM/YYYY
aaa	xxx
aaa	xxx

createEvent

EventHub

Search for events by name, tag, or club

All Clubs

All Events

My Account

Event Name:

Club: Club 1 Club 2

Tags: ☐ tech ☐ music ☐ law ☐ STEM ☐ careers ☐ sport

Date: DD/MM/YYYY

Time: from 00:00 AM to 00:00 AM PM PM

Location:

Price: \$

Event Description:

Create Event

eventManagement

EventHub

Search for events by name, tag, or club

All Clubs

All Events

My Account

abc

Edit Delete

Add Picture

Club: def
Organiser: Me
Tags: xyz
Date: DD/MM/YYYY
Time: xxxxx PM
Location: xxx
Price: \$xx
Event Description: abcdefg

RSVP users

	User Email
1	xxx
2	def
3	xxx
4	xxx
5	xxx
6	xxx
7	xxx
8	xxx
9	xxx
10	xxx
11	xxx

8. Appendix A: Glossary

- RSVP: A user indication of attendance
- EventHub: Name of the event management application
- MVC: Model-View-Controller
- JWT: JSON Web Token
- REST API: Representational State Transfer Application Programming Interface
- ACID: An acronym standing for atomicity, consistency, isolation, and durability. These properties are all desirable in a database system, and are all closely tied to the notion of a transaction.

9. Appendix B: Analysis Models

Page Structure

-> means this goes to another page
This is organized chronologically, not by importance.
'Details' pages can be made roughly from the same template.
I haven't talked about the signup/login pages.

Landing Page Elements

- Signup button (-> [signup page](#) -> [homepage](#))
- Login button (-> [login page](#) -> [homepage](#))

Homepage Elements

- Banner (shows banner pages and account)

Student Specific Elements:

- Carousel view of the top 5 earliest RSVP'd events
- Carousel view of the top 5 recommended events

Club Organiser Specific Elements:

- Carousel view of the top 5 earliest events from organiser's club/s
- + Student Specific Elements

Banner Elements (accessible from all pages except landing, login, and signup pages)

- Search bar
- Escape hatch (logo -> [homepage](#))
- View Events (-> [all events page](#))
- View Clubs (-> [all clubs page](#))
- Account dropdown menu (this can be a name instead of/in addition to a picture)

Account Dropdown Menu Elements

- My RSVP'd Events
- Logout

Club Organiser Specific Elements:

- View My Clubs (-> [my clubs page](#))

System Administrator Specific Elements

- Administrator Options (-> [administrator options page](#))

My Clubs Page Elements

- List of clubs assigned to the organiser (clicking on a club -> [club details view](#))

Administrator Options Page Elements

- Table of all users (clicking on a user -> [user details page](#))
- + additional admin options

User Details Page Elements (only accessible by system administrators)

- Name
- Email address
- Date joined
- + any other attributes
- Delete user button (-> [administrator options page](#))

All Events Page Elements

- Table of all events (clicking on an event -> [event details page](#))
- Filter + sorting + search options
 - Default (sorted by date: closest)

Event Details Page Elements

- Name
- Description
- Club (click on club -> [club details page](#))
- Date
- + any other attributes

Club Organiser and System Administrator Specific Elements:

- Edit event details (name, description, date, place, etc.)
- Delete club

All Clubs Page Elements (functionally identical to the [all events page](#) but can/should be visually different)

- Table of all clubs (clicking on a club -> [club details page](#))
- Filter + sorting + search options

Club Details Page Elements

- Name
- Description
- Organiser (user that is organiser)
- Tag (type of club)
- Date made
- List of events (clicking on event -> [event details page](#))

Club Organiser + Specific Elements:

- Manage club (edit name, description, tag, etc.)
- Delete club

10. Appendix C: Milestone 2

10.1 New Requirements in Milestone 2

No was written in the first sprint, so all existing code, tests, and functionality was done in Milestone 2. This sprint covered the basic functionality of the program and the minimum functional requirement (MVP). Because of this, there were few additions to the specifications of the project that were defined in Milestone 1. However, some updates and adjustments were necessary to help us better achieve the MVP goal.

- Updated from dynamically displaying dashboard information to redirecting users to separate dashboards based on their user type (Student, Organiser, Admin). This was done for clarity and to shorten development time.
- Created a new user story for email notifications to alert users about events. *(Not completed in this sprint, planned for Sprint 3.)*
- Created a user story to allow users to share events with others. *(Not completed in this sprint, planned for a Sprint 3.)*
- Created a club creation user story, which was overlooked in Milestone 1.
- Removed the separate task for assigning a club organiser, as the club creator is automatically designated as the organiser.

10.2 Description of New Functionality in Milestone 2

The minimum requirement functionality required for EventHub was achieved, which included new features such as user authentication and registration, the ability for organisers to create clubs and events, and the ability for students view, search, and RSVP to events. Added functionality in this Milestone also included the creation of interface components such as navigation bars, event panels, and buttons for RSVP, sharing, and deletion of events. Finally, API endpoints were created to manage data such as users, events, and RSVPs, and input validation and confirmation modals were established to make sure the application was stable and secure.

10.3 Code Files Added in Sprint 2

src\main\java\au\edu\rmit\sept\webapp\config\AuthInterceptorConfig.java

src\main\java\au\edu\rmit\sept\webapp\controller\UserController.java

src\main\java\au\edu\rmit\sept\webapp\controller\EventController.java

src\main\java\au\edu\rmit\sept\webapp\controller\HomeController.java

src\main\java\au\edu\rmit\sept\webapp\model\User.java

src\main\java\au\edu\rmit\sept\webapp\model\Event.java

src\main\java\au\edu\rmit\sept\webapp\model\RSVP.java

src\main\java\au\edu\rmit\sept\webapp\repository\RSVPRepository.java

src\main\java\au\edu\rmit\sept\webapp\repository\UserRepository.java

src\main\java\au\edu\rmit\sept\webapp\repository\EventRepository.java

src\main\java\au\edu\rmit\sept\webapp\service\EventService.java

src\main\java\au\edu\rmit\sept\webapp\service\RSVPService.java

src\main\java\au\edu\rmit\sept\webapp\service\UserService.java

src\main\java\au\edu\rmit\sept\webapp\service\AuthInterceptor.java

src\main\resources\static\js\toggleViewMore.js

src\main\resources\static\css\landing-page.css

src\main\resources\static\css\navbar.css

src\main\resources\static\css\style.css

src\main\resources\static\css\table.css

src\main\resources\templates\delete-page.html

src\main\resources\templates\event-details.html

src\main\resources\templates\event-edit.html

src\main\resources\templates\event-form.html

src\main\resources\templates\event-list.html

src\main\resources\templates\event-success.html

src\main\resources\templates\landing-page.html
src\main\resources\templates\organiser-dashboard.html
src\main\resources\templates\rsvp-events.html
src\main\resources\templates\sign-in.html
src\main\resources\templates\sign-up.html
src\main\resources\templates\student-dashboard copy.html
src\main\resources\templates\user-delete.html
src\main\resources\templates\user-details.html
src\main\resources\templates\user-list.html
src\main\resources\templates\admin-dashboard.html
src\main\resources\templates\club-list.html
src\main\resources\templates\dashboard.html
src\main\resources\templates\fragments\eventTable.html
src\main\resources\templates\fragments\navbar.html
src\main\resources\templates\fragments\userTable.html

10.4 Test Files Added in Sprint 2

src\test\java\au\edu\rmit\sept\webapp\Club\ClubRepositoryTest.java
src\test\java\au\edu\rmit\sept\webapp\Club\ClubServiceTest.java
src\test\java\au\edu\rmit\sept\webapp\Club\ClubHelperTest.java
src\test\java\au\edu\rmit\sept\webapp\Event\EventRepositoryTest.java
src\test\java\au\edu\rmit\sept\webapp\Event\EventServiceTest.java
src\test\java\au\edu\rmit\sept\webapp\Event\EventHelperTest.java
src\test\java\au\edu\rmit\sept\webapp\RSVP\RSVPRepositoryTest.java
src\test\java\au\edu\rmit\sept\webapp\RSVP\RSVPServiceTest.java
src\test\java\au\edu\rmit\sept\webapp\RSVP\RSVPHelperTest.java
src\test\java\au\edu\rmit\sept\webapp\StudentDashboard\RSVPServiceTest.java

src\test\java\au\edu\rmit\sept\webapp\User\UserRepositoryTest.java

src\test\java\au\edu\rmit\sept\webapp\User\UserServiceTest.java

src\test\java\au\edu\rmit\sept\webapp\User\UserHelperTest.java

10.5 Other Files

Meeting minutes, additional communication document, sprint planning, sprint retro, and acceptance test documentation and execution are stored within their own documents.